

SEMANA 2 — DÍA 5

Refinamiento del Script 1 hacia el ENTREGABLE 2

Objetivo del día:

Que cada equipo avance de manera autónoma para robustecer su **Script 1 FINAL**, integrando:

- Validación real con expresiones regulares
- Limpieza y normalización completa
- Modularidad clara
- Uso correcto de `os` y `sys`
- Guardado de datos limpios
- Documentación profesional

Al finalizar la sesión, cada equipo debe tener un **avance sólido del Script 1 FINAL**, listo para continuar la siguiente semana.

TEORÍA

1. ¿Qué es el Script 1 FINAL?

A diferencia del Script 1 v0 (que ya lograron), el Script 1 FINAL debe:

- Ser **modular** (mínimo 2 módulos, idealmente 4–5)
- Incluir **validación real con regex**
- Tener **limpieza completa y consistente**
- Usar **`os`** para crear carpetas y manejar rutas
- Usar **`sys.argv`** si su API lo requiere
- Generar **archivos limpios** en `data/clean/`
- Estar **documentado** en README
- Ser **reproducible** desde la terminal

Idea clave: El Script 1 FINAL es la versión profesional del flujo de datos de su API.

2. ¿Qué es `sys.argv` y para qué sirve?

`sys.argv` permite recibir parámetros desde la terminal cuando ejecutas un script.

Ejemplo:

```
python main.py mexico 2024-04-15
```

Dentro del script:

```
import sys
print(sys.argv)
```

Salida:

```
['main.py', 'mexico', '2024-04-15']
```

Esto permite que el usuario del script elija datos específicos para la consulta, como pueden ser:

- ciudad
- fecha
- ID
- país

- tipo de consulta
- endpoint

✓ ¿Cuándo usar sys.argv?

Usarlo si tu API **requiere parámetros variables**, por ejemplo:

- /pokemon/<nombre>
- /weather?city=<ciudad>
- /rates?date=<fecha>
- /characters/<id>

✓ ¿Cuándo NO es necesario?

Si tu API:

- siempre consulta el mismo endpoint
- no requiere parámetros
- no cambia según el usuario

✓ Aclaración importante

El uso de sys.argv puede implementarse en Script 1 o en Script 2.

Si tu API lo requiere, intégralo desde ya en Script 1.

Si tu API no lo necesita todavía, puedes implementarlo en los entregables 3 y 4 (Script 2).

3. Buenas prácticas para avanzar hacia el ENTREGABLE 2

- Separar responsabilidades por módulo
- No mezclar validación con limpieza
- No duplicar código
- Manejar errores en cada etapa
- Validar ANTES de limpiar
- Crear funciones pequeñas y claras
- Documentar cada función con docstrings
- Mantener nombres consistentes
- Probar el flujo completo desde main.py

4. Errores comunes que deben evitar HOY

- Regex demasiado permisivas o complejas
- Validaciones que no se usan
- Limpieza incompleta o inconsistente
- No aplanar estructuras anidadas
- No convertir tipos
- No manejar valores nulos
- No recibir parámetros con sys.argv cuando su API lo necesita
- No documentar el flujo en README

CHECKLIST (ENTREGABLE 2)

- ✓ Conexión a la API (api_client.py)
- ✓ Validación real con regex (validators.py)
- ✓ Limpieza y normalización (cleaner.py)
- ✓ Aplanamiento de estructuras anidadas
- ✓ Conversión de tipos
- ✓ Manejo de valores nulos
- ✓ Guardado de datos limpios (data/clean/)
- ✓ Uso de sys.argv si su API lo requiere
- ✓ Ejecución desde main.py
- ✓ Modularidad clara (mínimo 2 módulos, ideal 4–5)
- ✓ README actualizado con flujo del proyecto
- ✓ Commits de todos los integrantes

ACTIVIDAD EN CLASE

Cada equipo debe avanzar directamente hacia el **Script 1 FINAL**, realizando lo siguiente:

1. Ejecutar su flujo completo actual

- Verificar que no haya errores
- Verificar que el flujo sea correcto
- Verificar que los datos limpios se generen correctamente

2. Mejorar la validación con regex

- Agregar al menos **2 (de preferencia 3) patrones reales**
- Validar campos clave de su API
- Imprimir advertencias claras

Ejemplos útiles:

- IDs numéricos: `^\d+$`
- Fechas: `^\d{4}-\d{2}-\d{2}$`
- Textos: `^[A-Za-zÁÉÍÓÚáéíóúÑ\s]+$`

3. Completar la limpieza y normalización

- Convertir tipos
- Aplanar estructuras
- Normalizar nombres
- Manejar valores nulos
- Crear un diccionario limpio consistente

4. Evaluar si su API requiere sys.argv

Si sí → implementarlo hoy.

Si no → documentar por qué no es necesario (y se implementará en Script 2).

5. Asegurar modularidad

Revisar que:

- api_client.py solo conecte
- validators.py solo valide
- cleaner.py solo limpie
- utils.py tenga funciones auxiliares
- main.py orqueste todo

6. Subir avances a GitHub

- Commits claros
- Módulos actualizados
- Archivo limpio generado
- Todos los integrantes deben aparecer



TAREA (ENTREGABLE 2 – 26 de abril)

Cada equipo debe:

✓ Completar validaciones con regex

Al menos **3 patrones reales** aplicados a su API.

✓ Completar limpieza profunda

- Conversión de tipos
- Aplanamiento
- Normalización
- Manejo de nulos

✓ Integrar parámetros por CLI si su API lo requiere

Si no lo requiere → documentarlo en README.

✓ Documentar el flujo en README

Debe incluir:

- Cómo ejecutar el script
- Qué parámetros recibe (si aplica)
- Ejemplo de ejecución
- Explicación del flujo

✓ Subir avances a GitHub

- Módulos completos
- Flujo funcional
- Commits de todos